TikuOS Processes

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 4, 2026

Outline

1. Why Protothreads?
2. Process Data Structures
3. Creating a Process
4. Process Lifecycle
5. Protothread Mechanics
6. Events & Communication
7. Channels (Typed IPC)
8. Local Storage
9. Scheduler
10. Examples
11. Test Suite
12. Common Patterns
13. Summary

The Challenge: Concurrency on 2 KB RAM

Traditional RTOS threads:

- Each thread needs its own stack
- Typical minimum: 128–512 bytes/thread
- 4 threads = 512–2048 bytes of stack alone
- Impossible on 2 KB SRAM

Stack Overflow

No MMU means no guard pages. Stack overflow silently corrupts data.

Protothreads (TikuOS approach):

- **Stackless** — 2 bytes per thread
- Cooperative scheduling (no preemption)
- Event-driven: sleep until woken
- Continuation stored in a single `lc_t`
- 100+ protothreads on 2 KB RAM

Trade-off

No local variables across yields — use process-local storage instead.

Process Control Block

kernel/process/tiku_process.h

```
typedef struct tiku_process {
    struct tiku_process *next;
    const char          *name;
    PT_THREAD((*thread)(
        struct pt *,
        tiku_event_t,
        tiku_event_data_t));
    struct pt  pt;
    uint8_t    is_running;
    void      *local;
} tiku_process_t;
```

Field	Purpose
`next`	Linked list of active processes
`name`	Human-readable label
`thread`	Protothread function pointer
`pt`	Continuation state (2 bytes)
`is_running`	Active flag
`local`	Per-process storage pointer

Footprint

~12 bytes per process on MSP430 (16-bit pointers).

Event System

System events:

```
TIKU_EVENT_INIT        0x01
TIKU_EVENT_EXIT        0x02
TIKU_EVENT_CONTINUE    0x03
TIKU_EVENT_POLL        0x04
TIKU_EVENT_EXITED      0x05
TIKU_EVENT_FORCE_EXIT  0x06
TIKU_EVENT_USER        0x10
TIKU_EVENT_TIMER       0x88
```

Custom events start at `TIKU_EVENT_USER`:

```
#define EVENT_NEW_DATA \
    (TIKU_EVENT_USER + 0)
#define EVENT_BTN_PRESS \
    (TIKU_EVENT_USER + 1)
```

Event queue (circular buffer):

```
struct event_item {
    tiku_event_data_t data;
    struct tiku_process *p;
    tiku_event_t ev;
};

static struct event_item
    queue[TIKU_QUEUE_SIZE]; // 32
static volatile uint8_t q_head;
static volatile uint8_t q_len;
```

- 32 entries (power of 2)
- ISR-safe (atomic sections)
- Broadcast: `p = NULL`

Minimal Process — Step by Step

```c
/* 1. Declare the process */
TIKU_PROCESS(my_process, "My Process");

/* 2. Define the protothread */
TIKU_PROCESS_THREAD(my_process, ev, data)
{
    TIKU_PROCESS_BEGIN();           /* must be first */

    /* one-time init code here */
    tiku_common_led1_init();

    while (1) {
        TIKU_PROCESS_WAIT_EVENT(); /* sleep until any event */

        if (ev == TIKU_EVENT_TIMER) {
            tiku_common_led1_toggle();
        }
    }

    TIKU_PROCESS_END();            /* must be last */
}

/* 3. Register for autostart */
TIKU_AUTOSTART_PROCESSES(&my_process);
```

Process Declaration Variants

No local storage

```
TIKU_PROCESS(name, "Human Name");
/* local pointer = NULL */
```

With local storage (manual cast)

```
TIKU_PROCESS_WITH_LOCAL(name, "Human Name", struct my_state);
/* Access: TIKU_LOCAL(struct my_state)->field */
```

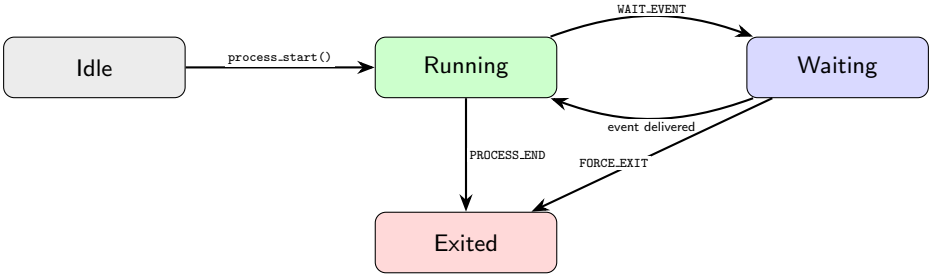
With typed accessor (recommended)

```
TIKU_PROCESS_TYPED(name, "Human Name", struct my_state);
/* Access: name_local()->field   (type-safe, no cast) */
```

Local Storage

- Statically allocated, zero-initialized (BSS)
- Persists across yields and process restarts
- Solves the "no stack variables across yields" limitation

Lifecycle Overview



Start:

- `tiku_process_start()` adds to list
- Posts `TIKU_EVENT_INIT`
- Sets `is_running = 1`

Exit:

- Automatic on `PROCESS_END`
- Or `PROCESS_EXIT()` (early)
- Or `FORCE_EXIT` event (kill)
- Removed from linked list

Key Process API

Function / Macro	Description
`tiku_process_start(p, data)`	Add process to list, post INIT event
`tiku_process_exit(p)`	Remove process from list, mark inactive
`tiku_process_post(p, ev, d)`	Post event to process (or broadcast if `p = NULL`)
`tiku_process_poll(p)`	Post POLL event (safe from ISR)
`tiku_process_run()`	Dequeue and dispatch one event
`TIKU_PROCESS_BEGIN()`	Start of protothread body
`TIKU_PROCESS_END()`	End of protothread body
`TIKU_PROCESS_WAIT_EVENT()`	Yield until any event
`TIKU_PROCESS_WAIT_EVENT_UNTIL(c)`	Yield until condition c is true
`TIKU_PROCESS_YIELD()`	Cooperative yield
`TIKU_PROCESS_EXIT()`	Early exit from protothread
`TIKU_THIS()`	Pointer to current process

How Protothreads Work

What you write:

```c
TIKU_PROCESS_THREAD(led, ev, data)
{
    TIKU_PROCESS_BEGIN();

    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(
            ev == TIKU_EVENT_TIMER);
        tiku_common_led1_toggle();
    }

    TIKU_PROCESS_END();
}
```

What the compiler sees:

```c
char thread_led(struct pt *pt,
    tiku_event_t ev, ...) {
  char PT_YIELD_FLAG = 1;
  switch (pt->lc) {
  case 0:  /* PROCESS_BEGIN */

    while (1) {
  case __LINE__:  /* WAIT_EVENT */
      if (!(ev == TIKU_EVENT_TIMER))
        return PT_YIELDED;
      toggle_led();
    }

  }  /* PROCESS_END */
  pt->lc = 0;
  return PT_ENDED;
}
```

Important Rule

No local variables across yield points. The switch/case jump skips initializers. Use process-local storage or `static` variables instead.

Protothread Return Codes

Code	Value	Meaning
`PT_WAITING`	0	Blocked on a condition
`PT_YIELDED`	1	Voluntarily yielded, ready to resume
`PT_EXITED`	2	Early exit (`PT_EXIT`)
`PT_ENDED`	3	Reached `PT_END` naturally

The scheduler uses these to manage process lifecycle:

- `PT_WAITING` or `PT_YIELDED` — process stays active
- `PT_EXITED` or `PT_ENDED` — scheduler auto-calls `tiku_process_exit()`
- `PT_SCHEDULE(f)` macro: true if thread still running (`f < PT_EXITED`)

Posting Events

Post to a single process

```
#define EVENT_NEW_DATA (TIKU_EVENT_USER + 0)

/* Post with data payload (cast to void*) */
tiku_process_post(&consumer, EVENT_NEW_DATA, (void *)(intptr_t)value);
```

Broadcast to all processes

```
tiku_process_post(TIKU_PROCESS_BROADCAST, EVENT_HEARTBEAT, NULL);
/* TIKU_PROCESS_BROADCAST is NULL --- iterates entire process list */
```

Poll from ISR

```
/* Inside an interrupt handler */
tiku_process_poll(&sensor_process);
/* Posts TIKU_EVENT_POLL, wakes the process */
```

ISR Safety

All post/poll functions use atomic sections — safe to call from interrupt context. Queue holds 32 events; returns 0 if full.

Receiving Events

Wait for any event

```
TIKU_PROCESS_WAIT_EVENT();
/* ev and data are set when process resumes */
```

Wait for a specific event

```
TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
/* Ignores all other events, only wakes on TIMER */
```

Event dispatch loop pattern

```
while (1) {
    TIKU_PROCESS_WAIT_EVENT();

    if (ev == TIKU_EVENT_TIMER) {
        /* handle timer */
    } else if (ev == EVENT_NEW_DATA) {
        int value = (int)(intptr_t)data;
        /* handle data */
    } else if (ev == TIKU_EVENT_EXIT) {
        /* cleanup and exit */
        TIKU_PROCESS_EXIT();
    }
}
```

Channel: Type-Safe Message Passing

Declare a channel

```
struct sensor_msg {
    uint16_t seq;
    uint16_t value;
};

/* Creates: sensor_ch, sensor_ch_init(), sensor_ch_put(), sensor_ch_get() */
TIKU_CHANNEL_DECLARE(sensor_ch, struct sensor_msg, 4);  /* depth = 4 */
```

Producer

```
struct sensor_msg msg;
msg.seq   = seq++;
msg.value = read_adc();

if (sensor_ch_put(&msg)) {
    /* notify consumer */
    tiku_process_post(&consumer,
        EVENT_DATA_READY, NULL);
}
```

Consumer

```
TIKU_PROCESS_WAIT_EVENT_UNTIL(
    ev == EVENT_DATA_READY);

struct sensor_msg msg;
while (sensor_ch_get(&msg)) {
    /* process msg.seq, msg.value */
}
```

Channel Internals

Channel Data Structure & API

kernel/process/tiku_process.h

```
typedef struct tiku_channel {
    uint8_t *buf;
    uint8_t  msg_size;
    uint8_t  capacity;
    volatile uint8_t head;
    volatile uint8_t count;
} tiku_channel_t;
```

- Caller-provided static buffer
- `volatile` head/count for ISR safety
- memcpy-based put/get

Function	Description
`_init()`	Wire buffer, set head/count=0
`_put(msg)`	Copy msg to tail; returns 1/0
`_get(out)`	Copy head to out; returns 1/0
`is_empty()`	True if count == 0
`free()`	Remaining capacity

`TIKU_CHANNEL_DECLARE` generates typed inline wrappers — no raw pointer casts needed at the call site.

Process-Local Storage

Declaring process with local storage

```
struct blink_state {
    uint16_t count;
    uint8_t  led_on;
};

TIKU_PROCESS_TYPED(blink_proc, "Blink", struct blink_state);
```

Using local storage inside protothread

```
TIKU_PROCESS_THREAD(blink_proc, ev, data)
{
    /* Get typed pointer ABOVE PROCESS_BEGIN (stack variable) */
    struct blink_state *s = blink_proc_local();

    TIKU_PROCESS_BEGIN();

    s->count = 0;
    s->led_on = 0;

    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        s->count++;
        s->led_on = !s->led_on;
    }
```

Scheduler Loop

kernel/scheduler/tiku_sched.c

```
void tiku_sched_loop(void)
{
    /* Start autostart processes */
    tiku_autostart_start(
        tiku_autostart_processes);

    while (sched_state ==
           TIKU_SCHED_RUNNING) {

        /* 1. Drain all pending events */
        while (tiku_sched_run_once())
            ;

        /* 2. If truly idle, call hook */
        /* (enter low-power mode) */
        if (!tiku_sched_has_pending()
            && idle_hook) {
            idle_hook();
        }
    }
}
```

Execution flow:

1. Boot calls `tiku_sched_loop()`
2. Autostart processes are started
3. Loop: drain event queue
4. Each `run_once()` dequeues one event and dispatches it
5. When idle, call hook (LPM)
6. ISR posts event → CPU wakes
7. Repeat

Idle Hook

```
tiku_sched_set_idle_hook(
    enter_lpm3);
/* Called when no events pending */
```

Autostart

Register processes for autostart

```
/* Single process */
TIKU_AUTOSTART_PROCESSES(&blink_process);

/* Multiple processes */
TIKU_AUTOSTART_PROCESSES(&producer, &consumer, &logger);
```

- Creates a NULL-terminated array of process pointers
- Scheduler iterates and calls `tiku_process_start()` on each
- Every process receives `TIKU_EVENT_INIT` as its first event
- Processes are started in declaration order

Manual Start (without autostart)

```
/* Start a process at runtime */
tiku_process_start(&my_process, NULL);

/* Start via scheduler wrapper */
tiku_sched_start(&my_process, NULL);
```

Example 1: LED Blink

examples/01_blink/blink.c

```
static struct tiku_timer blink_timer;

TIKU_PROCESS(blink_process, "Blink");

TIKU_PROCESS_THREAD(blink_process, ev, data)
{
    TIKU_PROCESS_BEGIN();

    tiku_common_led1_init();
    tiku_timer_set_event(&blink_timer, TIKU_CLOCK_SECOND);

    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_common_led1_toggle();
        tiku_timer_reset(&blink_timer);   /* drift-free restart */
    }

    TIKU_PROCESS_END();
}

TIKU_AUTOSTART_PROCESSES(&blink_process);
```

Example 2: Dual Blink (Two Processes)

examples/02_dual_blink/dual_blink.c

```
static struct tiku_timer red_timer, green_timer;

TIKU_PROCESS(red_blink,   "Red Blink");      /* 500 ms */
TIKU_PROCESS(green_blink, "Green Blink");     /* 1.5 s  */

TIKU_PROCESS_THREAD(red_blink, ev, data) {
    TIKU_PROCESS_BEGIN();
    tiku_common_led1_init();
    tiku_timer_set_event(&red_timer, TIKU_CLOCK_SECOND / 2);
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_common_led1_toggle();
        tiku_timer_reset(&red_timer);
    }
    TIKU_PROCESS_END();
}

TIKU_PROCESS_THREAD(green_blink, ev, data) {
    TIKU_PROCESS_BEGIN();
    tiku_common_led2_init();
    tiku_timer_set_event(&green_timer, (3 * TIKU_CLOCK_SECOND) / 2);
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_common_led2_toggle();
```

Example 3: Producer–Consumer

examples/04_multi_process/multi_process.c (simplified)

```c
#define EVENT_NEW_DATA (TIKU_EVENT_USER + 0)

TIKU_PROCESS(producer, "Producer");
TIKU_PROCESS(consumer, "Consumer");

TIKU_PROCESS_THREAD(producer, ev, data) {
    static uint16_t counter = 0;
    TIKU_PROCESS_BEGIN();
    tiku_timer_set_event(&prod_timer, TIKU_CLOCK_SECOND);
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_process_post(&consumer, EVENT_NEW_DATA,
                          (void *)(intptr_t)counter++);
        tiku_timer_reset(&prod_timer);
    }
    TIKU_PROCESS_END();
}

TIKU_PROCESS_THREAD(consumer, ev, data) {
    TIKU_PROCESS_BEGIN();
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == EVENT_NEW_DATA);
        int value = (int)(intptr_t)data;
        /* process value */
```

Example 4: Broadcast

examples/07_broadcast/broadcast.c (simplified)

```c
#define EVENT_HEARTBEAT (TIKU_EVENT_USER + 0)

TIKU_PROCESS(controller, "Controller");
TIKU_PROCESS(listener_a, "Listener A");
TIKU_PROCESS(listener_b, "Listener B");

TIKU_PROCESS_THREAD(controller, ev, data) {
    TIKU_PROCESS_BEGIN();
    tiku_timer_set_event(&hb_timer, TIKU_CLOCK_SECOND);
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_process_post(TIKU_PROCESS_BROADCAST, EVENT_HEARTBEAT, NULL);
        tiku_timer_reset(&hb_timer);
    }
    TIKU_PROCESS_END();
}

TIKU_PROCESS_THREAD(listener_a, ev, data) {
    TIKU_PROCESS_BEGIN();
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == EVENT_HEARTBEAT);
        tiku_common_led1_toggle();
    }
    TIKU_PROCESS_END();
}
```

Example 5: Channel IPC

examples/09_channel/channel.c (simplified)

```
struct sensor_msg { uint16_t seq; uint16_t value; };
TIKU_CHANNEL_DECLARE(sensor_ch, struct sensor_msg, 4);
#define EVENT_DATA_READY (TIKU_EVENT_USER + 0)

TIKU_PROCESS_THREAD(sensor_proc, ev, data) {
    static uint16_t seq = 0;
    TIKU_PROCESS_BEGIN();
    sensor_ch_init();
    tiku_timer_set_event(&sensor_timer, TIKU_CLOCK_SECOND / 2);
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        struct sensor_msg msg = { seq++, seq * 10 };
        if (sensor_ch_put(&msg))
            tiku_process_post(&display_proc, EVENT_DATA_READY, NULL);
        tiku_timer_reset(&sensor_timer);
    }
    TIKU_PROCESS_END();
}

TIKU_PROCESS_THREAD(display_proc, ev, data) {
    TIKU_PROCESS_BEGIN();
    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == EVENT_DATA_READY);
        struct sensor_msg msg;
```

Test Configuration

tests/tiku_test_config.h

```
#define TEST_ENABLE 1

/* Process test flags */
#define TEST_PROCESS_LIFECYCLE      1
#define TEST_PROCESS_EVENTS         1
#define TEST_PROCESS_YIELD          1
#define TEST_PROCESS_BROADCAST      1
#define TEST_PROCESS_POLL           1
#define TEST_PROCESS_QUEUE          1
#define TEST_PROCESS_LOCAL          1
#define TEST_PROCESS_BROADCAST_EXIT 1
#define TEST_PROCESS_GRACEFUL_EXIT  1
#define TEST_PROCESS_CURRENT_CLEARED 1
```

Run on target:

```
# Enable flags, then:
make flash MCU=msp430fr5969
make monitor
```

10 test cases covering:

- Process start/exit lifecycle
- Event posting and reception
- Cooperative yielding phases
- Broadcast to multiple processes
- Poll from ISR context
- Queue capacity and overflow
- Local storage persistence
- Broadcast exit safety
- Graceful vs force exit
- Current process clearing

Test Cases Overview

#	Test Flag	What It Verifies
1	`LIFECYCLE`	start posts INIT; END auto-exits
2	`EVENTS`	custom event posting and reception
3	`YIELD`	multi-phase yield with continuation
4	`BROADCAST`	single post reaches all processes
5	`POLL`	`tiku_process_poll()` delivers POLL event
6	`QUEUE`	empty/full/length/space; overflow returns 0
7	`LOCAL`	WITH_LOCAL and TYPED storage persists across yields
8	`BROADCAST_EXIT`	process exiting mid-broadcast doesn't corrupt list
9	`GRACEFUL_EXIT`	EXIT event: process can do cleanup; FORCE_EXIT kills
10	`CURRENT_CLEARED`	`TIKU_THIS()` is NULL outside dispatch

Pattern Reference

Periodic timer

```c
static struct tiku_timer t;
/* in PROCESS_THREAD: */
tiku_timer_set_event(&t,
    TIKU_CLOCK_SECOND);
while (1) {
    TIKU_PROCESS_WAIT_EVENT_UNTIL(
        ev == TIKU_EVENT_TIMER);
    /* do work */
    tiku_timer_reset(&t);
}
```

Graceful shutdown

```c
while (1) {
    TIKU_PROCESS_WAIT_EVENT();
    if (ev == TIKU_EVENT_EXIT) {
        /* cleanup */
        TIKU_PROCESS_EXIT();
    }
    /* normal work */
}
```

ISR to process

```c
/* In ISR: */
tiku_process_poll(&my_proc);

/* In process: */
TIKU_PROCESS_WAIT_EVENT_UNTIL(
    ev == TIKU_EVENT_POLL);
/* handle hardware event */
```

Multi-event dispatch

```c
while (1) {
    TIKU_PROCESS_WAIT_EVENT();
    switch (ev) {
    case TIKU_EVENT_TIMER:
        handle_timer(); break;
    case EVENT_BTN:
        handle_button(); break;
    case TIKU_EVENT_EXIT:
        TIKU_PROCESS_EXIT();
    }
}
```

Summary

Process Model

- Stackless protothreads (2 bytes each)
- Event-driven cooperative scheduling
- 32-entry circular event queue
- ISR-safe posting and polling
- Autostart mechanism
- Broadcast events
- Per-process local storage

Communication

- Direct events: one-to-one
- Broadcast: one-to-all
- Channels: typed message queues
- Poll: ISR to process notification

Testing

10 test cases covering lifecycle, events, yield, broadcast, poll, queue, local storage, exit safety.

Thank You

TikuOS Processes

Protothreads & Events for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/process/tiku_process.h`
Examples:	`examples/01_blink/` ... `examples/09_channel/`
Tests:	`tests/test_process.c`